

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c\agent-a0d5d4bff0f9aab86.jsonl`
- SHA-256: `a1518448e8371eafe1b9472e85ec8cdc5d9355ad78043752304c92b436b6903f`
- Source modified: `2026-06-18T01:21:03+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_34138e24-10c`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:17:51.748Z)

You are the merge gatekeeper. Here are 4 adversarial reviews (JSON) of a small bug-fix PR for badminton-highlight-indexer:

```
[
  {
    "lens": "correctness & semantics",
    "summary": "The fix is correct and safe to merge. _is_fusion_schema
(backend/eval/ablation.py:386-395) keys on shuttle+person presence with audio kept optional,
exactly matching the two real schemas. I verified against the actual golden corpus (output/, 11
files): exactly 6 classify as fusion (adarsh_avi_singles, gbaaddy, kushagra_singles,
largetest_doubles, mahadevpura_singles, testlarge_short) and 5 as lean velocity-windowing
(Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2). The filter yields exactly
n=6, so the litmus's pinned numbers hold: test_litmus_reproduces_gate_a RAN (not skipped) and
PASSED ? ?F1?0.074, 6/0 wins, p?0.031, CI [+0.042,+0.104], seg ratios 1.01/1.68. All 29 ablation
tests pass; sibling fusion_compare/audio_features tests (12) pass; ruff + mypy clean. Audio-
optional tolerance is preserved end-to-end (a shuttle+person file with no audio dict loads with
audio columns defaulted to 0.0). No genuine fusion file is wrongly skipped (real files are
schema-homogeneous, so all(...) is safe) and no lean file is wrongly kept. _golden_present
(tests/test_ablation.py:408-422) now counts only fusion-schema files, correctly preventing a
future under-6 fusion corpus from running a mis-sized litmus. No must-fix found.",
    "findings": [
      {
        "severity": "none",
        "issue": "_is_fusion_schema uses all(...) over candidates, so a fusion file with even
ONE cue-less row would be silently SKIPPED rather than crashed. This is the intended fail-safe
behavior, and I confirmed every real fusion file in output/ is schema-homogeneous (all rows
carry shuttle+person+audio), so no real file is wrongly skipped today. Noting only as a latent
semantic assumption.",
        "location": "backend/eval/ablation.py:395",
        "suggestion": "No change needed. The all(...)+skip behavior is strictly safer than the
previous unconditional KeyError crash. If heterogeneous-row fusion files ever appear, the file
is skipped (loud INFO log) rather than crashing the run."
      },
      {
        "severity": "none",
        "issue": "Sibling loaders fusion_compare.load_videos
(backend/eval/fusion_compare.py:46-48) and fusion_audio.load_videos_with_audio still read
c[\"shuttle\"]/c[\"person\"] unconditionally and would KeyError on the same mixed corpus.
golden_fixtures.py:352 already documents this exact hazard. This is pre-existing and explicitly
outside this commit's scope (which targets only the ablation litmus), so it does not block merge
? flagging for awareness only.",
        "location": "backend/eval/fusion_compare.py:46",
        "suggestion": "Out of scope for this bug-fix. If fusion_compare/fusion_audio are ever
run against the mixed golden corpus they will crash the same way; a follow-up could factor
```

`_is_fusion_schema` into a shared filter. Not required for this PR."

```
    }
  ],
  "blocks_merge": false
},
{
  "lens": "edge cases & robustness",
  "summary": "Clean, narrowly-scoped fix. _is_fusion_schema gates load_videos so a mixed corpus (full-fusion + leaner velocity-windowing golden_features.json) loads the ablatable subset instead of crashing on KeyError: 'shuttle'. Verified END-TO-END on the REAL golden substrate (not just mocks): output/ here actually holds 11 golden files = 6 full-fusion + 5 lean (Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2). load_videos() cleanly returns the 6 fusion videos, and the golden litmus (which RAN, did not skip) reproduces the recorded Gate-A numbers exactly:  $\text{deltaF1}=0.0739$ , 6W/0L,  $p=0.0312$ ,  $\text{CI}=[0.042, 0.104]$ ,  $\text{seg}=1.01/1.68$ . Confirmed old master crashes with KeyError('shuttle') on Badminton_BXH_2 and the fix removes it. Full repo suite: 848 passed, 5 skipped (unrelated owner-box golden-trajectory skips in test_served_gate_a_regression.py), ruff clean, mypy clean. JSONDecodeError IS a ValueError subclass, so _golden_present's (OSError, ValueError) catch correctly swallows missing-file + malformed-JSON without masking logic errors. All lens edge cases pass: empty candidate list -> not fusion (skipped); mixed-within-one-file (some cands full, some lean) -> all() returns False -> whole file skipped, no half-load; shuttle-present-but-person-missing -> skipped; missing audio still defaults to 0.0. No must-fix found; the issues below are pre-existing and untouched/improved by this PR.",
  "findings": [
    {
      "severity": "none",
      "issue": "VERIFIED (not a defect): the fix's central claim holds on the real n=11 mixed corpus. load_videos('output') returns exactly the 6 full-fusion videos; the litmus reproduces  $\text{deltaF1}=0.0739$ , 6W/0L,  $p=0.0312$ ,  $\text{CI}=[0.042, 0.104]$ ,  $\text{seg\_full}=1.014/\text{seg\_ablated}=1.675$  ? within every test tolerance. Old master crashes KeyError('shuttle') on the lean Badminton_BXH_2 file; this PR removes that crash. The golden litmus tests RAN (did not skip) here and passed.",
      "location": "backend/eval/ablation.py:386-428; tests/test_ablation.py:408-450",
      "suggestion": "None ? merge-worthy as-is. Confirmed via direct execution against the local golden substrate plus full suite (848 passed / 0 failed), ruff + mypy clean."
    },
    {
      "severity": "should-fix",
      "issue": "PRE-EXISTING latent fragility (not introduced, and partly IMPROVED by this PR): the litmus skip gate is _golden_present() >= 6 while test_litmus_reproduces_gate_a asserts `len(videos) == 6` exactly. If a 7th FULL-FUSION golden file is ever added, the gate stays True but load_videos returns 7 and the == 6 assertion fails (litmus turns red on more data). This PR actually fixes the opposite drift (lean files no longer inflate the count past 6 and trip a crashing litmus), but the >=6-vs-==6 mismatch on fusion-count growth remains.",
      "location": "tests/test_ablation.py:422 (>= 6) vs tests/test_ablation.py:433 (== 6)",
      "suggestion": "Out of scope for this fix, but consider asserting `len(videos) >= 6` (and slicing/using the recorded 6) or making the litmus corpus selection explicit, so adding a 7th fusion clip doesn't break the litmus. Not a blocker."
    },
    {
      "severity": "nit",
      "issue": "_golden_present() catches (OSError, ValueError) around json.load().get('candidates'). json.load on a valid-but-non-dict top-level JSON (e.g. a bare list) makes .get() raise AttributeError, which is NOT caught and would propagate out of the skip-gate. Realistic failure modes (missing file, malformed JSON) ARE handled correctly (JSONDecodeError is a ValueError subclass). This is test-only helper code gating a skipif, and a top-level-non-dict golden file would be a corrupt fixture.",
      "location": "tests/test_ablation.py:415-419",
      "suggestion": "Optionally broaden to `except (OSError, ValueError, AttributeError)` or guard `isinstance(obj, dict)` before .get(). Very low priority ? test-only, unlikely input."
    },
    {
      "severity": "none",
      "issue": "load_videos() still does an unguarded json.load (no try/except) and accesses c['shuttle'][fn] / c['person'][fn] directly, so a file that passes _is_fusion_schema but has a per-cue dict MISSING an individual column (e.g. shuttle present but lacking peak_velocity), or a
```

```

garbage/non-JSON *_golden_features.json in the dir, would still crash the loader/CLI. Both
behaviors are PRE-EXISTING on origin/master (verified via git show) and untouched by this
change; the schema guard only prevents the far more common whole-file lean-schema crash.",
    "location": "backend/eval/ablation.py:409-420",
    "suggestion": "No action required for this PR (not a regression). If desired later, a
defensive per-file try/except in load_videos that logs-and-skips would harden the CLI against a
stray malformed file."
}
],
"blocks_merge": false
},
{
    "lens": "test quality & regression faithfulness",
    "summary": "The fix adds an `_is_fusion_schema` guard so `load_videos` skips leaner
`windowing-only` golden_features.json files instead of crashing with KeyError: 'shuttle' on a
mixed corpus. I verified against the real `output/` corpus (exactly 11 files = 6 full-fusion + 5
lean): the fix loads exactly the 6 ablatable files, and both litmus tests RUN (not skipped) and
reproduce the recorded Gate-A numbers (?F1?0.074, 6/6 wins, p?0.031, CI?[0.042,0.104]). All 29
tests pass; ruff is clean. Both new tests genuinely pin the bug: I reconstructed the pre-fix
`load_videos` and confirmed it raises `KeyError: 'shuttle'` on the same mixed corpus, and
`_is_fusion_schema` doesn't exist in old code ? so
`test_load_videos_skips_windowing_only_schema` and
`test_is_fusion_schema_distinguishes_lean_and_full` would both fail on pre-fix code. Assertions
are meaningful (exact name-set equality {v0..v5}, single `gate_a` row,
`num_videos==len(videos)==6`). `_write_windowing_only` mirrors the real lean schema exactly
(verified its 7 keys match real mahadevpura_1/GX010128 files). The litmus path in the new test
exercises the same `run_ablation`+cue-column code path that crashed. Private-name access
`ablation._is_fusion_schema` is reasonable for an internal unit test. No must-fix found ? safe
to merge.",
    "findings": [
        {
            "severity": "none",
            "issue": "New test test_load_videos_skips_windowing_only_schema genuinely pins the bug:
reconstructed pre-fix load_videos raises KeyError: 'shuttle' on the same mixed corpus (6 full +
3 lean fixtures), so the test would error on old code.
test_is_fusion_schema_distinguishes_lean_and_full targets a function that does not exist pre-fix
(AttributeError on old code). Neither passes trivially.",
            "location": "tests/test_ablation.py:385-399, tests/test_ablation.py:373-382",
            "suggestion": "No change needed ? verified both tests fail against pre-fix code."
        },
        {
            "severity": "none",
            "issue": "Regression faithfulness holds on the real corpus. output/ contains exactly 6
full-fusion + 5 lean files; _golden_present() now counts only the 6 ablatable files and
load_videos returns exactly those 6, so test_litmus_reproduces_gate_a runs (not skipped) and
reproduces ?F1?0.074, 6/6, p?0.031. The earlier n=11 KeyError crash is gone.",
            "location": "backend/eval/ablation.py:412-416, tests/test_ablation.py:408-422",
            "suggestion": "No change needed ? litmus verified green on the present golden
substrate."
        },
        {
            "severity": "none",
            "issue": "`_write_windowing_only` mirrors the REAL lean schema exactly. Real lean
candidate rows carry keys {active_frame_count, chunk_index, end, mean_velocity, peak_velocity,
start, track_id_count}; the fixture emits precisely these 7 keys and no cue dicts. Assertions
are meaningful: exact name-set equality {v0..v5} would break if any lean file leaked in, and
num_videos==len(videos) (ablation.py:340) so ==6 is not a hardcoded constant.",
            "location": "tests/test_ablation.py:360-370, backend/eval/ablation.py:340",
            "suggestion": "No change needed."
        },
        {
            "severity": "nit",
            "issue": "`_is_fusion_schema` only guards top-level presence of shuttle/person, not the
inner columns the loader reads. If a file had shuttle/person dicts missing an inner key (e.g.
shuttle without 'duration'), line 420 `c['shuttle'][fn]` would still KeyError. Not a concern for

```

```

the current real corpus (verified all 6 full files contain every SHUTTLE/PERSON column), so
theoretical only.",
    "location": "backend/eval/ablation.py:395, backend/eval/ablation.py:420-422",
    "suggestion": "Optional: if malformed-but-present cue dicts ever appear, switch the
inner reads to c['shuttle'].get(fn, 0.0) or extend the predicate to check inner columns. Not
needed now."
},
{
    "severity": "nit",
    "issue": "No explicit test for the asymmetric partial case (a candidate with 'person'
present but 'shuttle' absent, or vice versa). The `all('shuttle' in c and 'person' in c ...)`
predicate handles it correctly (returns False ? skipped), but only the symmetric lean case is
pinned by a test.",
    "location": "tests/test_ablation.py:373-382",
    "suggestion": "Optional: add a one-line case to
test_is_fusion_schema_distinguishes_lean_and_full, e.g. assert
_is_fusion_schema([{'shuttle':{}}]) is False, to lock the partial-schema branch."
}
],
"blocks_merge": false
},
{
    "lens": "repo conventions & scope (style/docstrings/naming, scope correctness, commit-
message accuracy, unused imports/dead code/typing, logging level)",
    "summary": "Clean, well-scoped bug fix. The diff touches exactly the right 2 files
(backend/eval/ablation.py, tests/test_ablation.py), does NOT touch scripts/nightly_regression.py
or run_regression.py, and stacks nothing unrelated. I reproduced the behavior on the real mixed
corpus: output/ has 11 golden files, _is_fusion_schema partitions them into exactly 6 fusion + 5
windowing-only, load_videos loads exactly the original 6, and test_litmus_reproduces_gate_a
actually RAN (not skipped) and reproduced the recorded Gate-A numbers (?F1?0.074, 6/6 wins,
p?0.031) ? so the litmus is preserved. New docstrings match the file's dense prose style; naming
(_is_fusion_schema, _write_windowing_only) fits the neighbors; info-level log for an expected
skip is the correct level (not a warning). No new imports (Sequence/Dict/Any already imported at
line 54), no dead code; ruff clean, mypy clean on the changed module. Full suite 848 passed / 5
skipped ? exactly matching the commit message; ablation-module coverage 99% (only the __main__
guard line uncovered), well above the 70 floor. Every quantitative claim in the commit message
is accurate and not over-stated. No must-fix or should-fix issues found.",
    "findings": [
        {
            "severity": "nit",
            "issue": "Docstring/test-data count mismatch: the docstring says the synthetic mixed
corpus contains \"the 5 newer real clips\", but the test body writes only 3 windowing-only files
(mahadevpura_1, GX010128, Badminton_BXH_2). The '5' refers to the real owner-box corpus (which I
confirmed has 5 lean files); the synthetic test uses 3 as a representative sample. Test logic is
correct (it asserts exactly the 6 v-prefixed fusion files load), this is purely imprecise
wording.",
            "location": "tests/test_ablation.py:387",
            "suggestion": "Either change \"the 5 newer real clips\" to \"3 leaner clips (a
representative sample of the 5 on the owner box)\" or bump the loop to write 5 lean files for
exact parity with the real n=11 corpus. Cosmetic only ? does not affect correctness."
        },
        {
            "severity": "none",
            "issue": "Considered whether load_videos should wrap json.load in try/except to match
_golden_present's new (OSError, ValueError) guard, since a malformed real golden file would be
skipped by the gate but still crash load_videos. Confirmed this is pre-existing behavior
(load_videos never had try/except) and out of scope for this schema fix ? not introduced or
worsened by this change.",
            "location": "backend/eval/ablation.py:409-411",
            "suggestion": "No action needed for this PR."
        }
    ],
    "blocks_merge": false
}
]

```

Decide: verdict="merge" only if there is NO genuine must-fix that would break the litmus, cause a regression, or ship a real bug. Deduplicate overlapping findings across lenses. Be skeptical of findings that are actually non-issues (e.g. the ≥ 6 gate fragility is acceptable/pre-existing if the current corpus has exactly 6 fusion files and the litmus is intentionally pinned to $n=6$). List the real must_fixes (empty if none), should_fixes, and nits, with a crisp rationale.

2. assistant (2026-06-18T01:17:54.707Z)

I'll verify the key claims before deciding. Let me start by examining the actual code and corpus.

3. user (2026-06-18T01:17:56.131Z)

```
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
387         """True iff every candidate row carries the per-cue feature dicts the ablation
scorer reads
388         (`shuttle` + `person`; `audio` stays optional ? older JSONs predate it).
389
390         The velocity-windowing / distill path persists a LEANER ``*_golden_features.json``
row
391         (`start`/`end` + velocity stats, no cue dicts) for clips that never went through
fusion
392         feature extraction. ``load_videos`` SKIPS those rather than crash on
``c["shuttle"]`` ? so a
393         mixed corpus (some fusion files, some windowing-only) loads the ablatable subset
cleanly.
394         An empty candidate list is not ablatable either."""
395         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
396
397
398     def load_videos(features_dir: str) -> List[VideoCandidates]:
399         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
400         (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
401         (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
402         the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
403
404         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
405         R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
loads the
406         ablatable subset cleanly (see ``_is_fusion_schema``)."""
407         videos: List[VideoCandidates] = []
408         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
409             with open(path, encoding="utf-8") as f:
410                 d = json.load(f)
411                 cands = d["candidates"]
412                 if not _is_fusion_schema(cands):
413                     logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
414                                "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
415                                os.path.basename(path))
416                 continue
```

```

417         intervals = [(float(c["start"]), float(c["end"])) for c in cand]
418         features: Dict[str, List[float]] = {}
419         for fn in SHUTTLE_COLUMNS:
420             features[fn] = [float(c["shuttle"][fn]) for c in cand]
421         for fn in PERSON_COLUMNS:
422             features[fn] = [float(c["person"][fn]) for c in cand]
423         for fn in AUDIO_COLUMNS:
424             features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cand]
425         gts = [(float(a), float(b)) for a, b in d["gts"]]
426         videos.append(VideoCandidates(name=d["name"], intervals=intervals,
427                                     features=features, gts=gts))
428     return videos
429
430
431     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
432         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
433         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
434         (distinct from the static Gate-A litmus)."""
435         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
436         return experiment.Variant(name="full_learned", feature_names=cols,
437                                 threshold=threshold, min_crossings=min_crossings)
438
439

```

4. user (2026-06-18T01:17:56.744Z)

```

355         videos = ablation.load_videos(str(tmp_path))
356         assert len(videos) == 1
357         assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not
crashed
358
359
360     def _write_windowing_only(d: str, name: str) -> None:
361         """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema
the
362         R3/R4 corpus-growth path persists for clips that never went through fusion feature
363         extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts).
The
364         ablation loader must SKIP these, never crash on ``c["shuttle"]``.
365         cand = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
366                 "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
367         data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
368               "candidates": cand, "gts": [[0.0, 5.0]]}
369         with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
370             json.dump(data, f)
371
372
373     def test_is_fusion_schema_distinguishes_lean_and_full():
374         """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and
rejects
375         the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
376         full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
377         no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          #
audio optional
378         lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
379         assert ablation._is_fusion_schema(full) is True
380         assert ablation._is_fusion_schema(no_audio) is True
381         assert ablation._is_fusion_schema(lean) is False
382         assert ablation._is_fusion_schema([]) is False
383
384

```

```

385     def test_load_videos_skips_windowing_only_schema(tmp_path):
386         """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files
load; the
387         leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
388         and the litmus path runs end-to-end over the filtered ablatable subset."""
389         d = str(tmp_path)
390         _write_learned_corpus(d, n=6)                                # 6 full-fusion files
(v0..v5)
391         for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"): # 3 leaner real-clip
names
392             _write_windowing_only(d, name)
393             # only the fusion-schema files load ? the leaner ones never crash the loader
394             videos = ablation.load_videos(d)
395             assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
396             # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
397             report = run_ablation(videos, ablation.litmus_config())
398             assert [r.signal for r in report.rows] == ["gate_a"]
399             assert report.num_videos == 6
400
401
402         # ----- the
golden litmus
403
404         GOLDEN_DIR = os.path.join(
405             os.path.dirname(os.path.abspath(__file__)), "output")
406
407
408         def _golden_present() -> bool:
409             """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
410             columns). Counts only ablatable files ? leaner velocity-windowing files are skipped
by
411             ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
412             mixed-schema bug)."""
413             n = 0
414             for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
415                 try:
416                     with open(p, encoding="utf-8") as f:
417                         cands = json.load(f).get("candidates") or []
418                     except (OSError, ValueError):
419                         continue
420                     if ablation._is_fusion_schema(cands):
421                         n += 1
422             return n >= 6
423
424
425         @pytest.mark.skipif(not _golden_present(),
426                             reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
427         def test_litmus_reproduces_gate_a():
428             """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
429             result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
430             This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
431             where the golden substrate is absent; runs on the owner box where it is present."""
432             videos = ablation.load_videos(GOLDEN_DIR)
433             assert len(videos) == 6
434             cfg = ablation.litmus_config()
435             report = run_ablation(videos, cfg)
436             assert len(report.rows) == 1

```

```

437         row = report.rows[0]
438         assert row.signal == "gate_a"
439         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
440         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
441         assert row.sign_wins == 6 and row.sign_losses == 0
442         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
443         assert row.ci_low == pytest.approx(0.042, abs=0.01)
444         assert row.ci_high == pytest.approx(0.104, abs=0.01)
445         assert row.ci_excludes_zero is True
446         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
447         assert row.verdict == ablation.STRUCTURAL_PROTECTED
448         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
449         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
450         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
451
452
453     @pytest.mark.skipif(not _golden_present(),
454                         reason="golden output/*_golden_features.json absent")

```

5. assistant (2026-06-18T01:18:07.777Z)

The core logic is confirmed. Now let me verify the claims against the real corpus and run the tests. Let me check the git diff scope and the actual golden files.

6. user (2026-06-18T01:18:09.913Z)

```

59ae347 fix(eval): ablation litmus tolerates a mixed-schema golden corpus
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
(#205)
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
---DIFF STAT---
  backend/eval/ablation.py | 29 ++++++-----
  tests/test_ablation.py   | 57 ++++++-----
  2 files changed, 81 insertions(+), 5 deletions(-)

```

7. user (2026-06-18T01:18:11.154Z)

```

11
---FILES---
output/Badminton_BXH_2_golden_features.json
output/GX010128_golden_features.json
output/GX030094_golden_features.json
output/adarsh_avi_singles_golden_features.json
output/gbaaddy_golden_features.json
output/kushagra_singles_golden_features.json
output/largetest_doubles_golden_features.json
output/mahadevpura_1_golden_features.json
output/mahadevpura_2_golden_features.json
output/mahadevpura_singles_golden_features.json
output/testlarge_short_golden_features.json

```

8. assistant (2026-06-18T01:18:14.876Z)

The diff is exactly 2 files as claimed, and there are 11 golden files. Let me verify the partition (6 fusion + 5 lean) and run the actual tests to confirm the litmus reproduces.

9. user (2026-06-18T01:18:20.737Z)

```

FUSION (6):
  adarsh_avi_singles_golden_features.json
  gbaaddy_golden_features.json
  kushagra_singles_golden_features.json

```



```

    largetest_doubles_golden_features.json
    mahadevpura_singles_golden_features.json
    testlarge_short_golden_features.json
LEAN (5):
    Badminton_BXH_2_golden_features.json
    GX010128_golden_features.json
    GX030094_golden_features.json
    mahadevpura_1_golden_features.json
    mahadevpura_2_golden_features.json
load_videos -> 6 videos: ['adarsh_avi_singles', 'gbaaddy', 'kushagra_singles',
'largetest_doubles', 'mahadevpura_singles', 'testlarge_short']

```

10. assistant (2026-06-18T01:18:23.647Z)

Exactly 6 fusion + 5 lean, and `load\_videos` returns the 6 fusion videos. The partition matches all four reviews. Now let me confirm the litmus and full test suite pass, and verify the pre-fix crash claim.

11. user (2026-06-18T01:18:42.974Z)

```

..... [100%]
29 passed in 16.97s

```

12. assistant (2026-06-18T01:18:45.756Z)

All 29 ablation tests pass (litmus included, since the golden corpus is present here). Let me verify the litmus is actually running not skipped, confirm the pre-fix crash, and run the broader suite plus lint/type checks.

13. user (2026-06-18T01:18:52.326Z)

```

rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 1 item

```

```

tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED [100%]

```

```

===== 1 passed in 1.80s =====
=== PRE-FIX CRASH CHECK ===
PRE-FIX raises FileNotFoundError [Errno 2] No such file or directory:
'C:\tmp\prefix_ablation.py'

```

14. assistant (2026-06-18T01:18:55.408Z)

Litmus PASSED (not skipped) ? confirming the pinned numbers hold. The pre-fix check failed on a path issue (Git Bash `/tmp` vs Windows). Let me redo the pre-fix crash verification with a proper temp path.

15. user (2026-06-18T01:19:00.996Z)

```

Exit code 1
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    spec.loader.exec_module(m)
    ~~~~~^
File "<frozen importlib._bootstrap_external>", line 1023, in exec_module
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix\prefix_ablation_tmp.py", line 86, in <module>
    @dataclass(frozen=True)
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\dataclasses.py", line 1295,

```



```

..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 42%]
..... [ 50%]
..... [ 59%]
..... [ 67%]
.....SSS [ 75%]
SS..... [ 84%]
..... [ 92%]
..... [100%]
848 passed, 5 skipped in 64.42s (0:01:04)

```

20. user (2026-06-18T01:20:21.402Z)

```

=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file

```

21. assistant (2026-06-18T01:20:26.388Z)

Everything verifies:

- Full suite: 848 passed, 5 skipped (the 5 skips are unrelated owner-box golden-trajectory regression tests, exactly as review 2 claimed)
- Ruff: clean
- Mypy: success, no issues (the notes are pre-existing untyped-body advisories in unrelated modules)

Let me do a final check on the two `should-fix`-adjacent concerns: the `>=6` vs `==6` litmus assertion, and confirm the `\_golden\_present` exception handling. Both are flagged as pre-existing/acceptable, but let me confirm the `>=6` gate is sound given exactly 6 fusion files today.

22. user (2026-06-18T01:20:27.546Z)

```

330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),

```

23. assistant (2026-06-18T01:20:32.070Z)

`num\_videos == len(videos)` is computed, not hardcoded ? confirming review 3's point that `== 6` is not a magic constant but derived from the loaded set. The `>=6` gate with `==6` litmus assertion is the one substantive should-fix-adjacent point: today there are exactly 6 fusion files, so the gate and assertion agree. The fragility (adding a 7th *fusion* file turns the

litmus red) is real but pre-existing, latent, and the corpus is intentionally pinned to n=6. It does not block this fix.

I have verified everything needed. Let me record the decision.

24. user (2026-06-18T01:20:54.414Z)

Structured output provided successfully

25. assistant (2026-06-18T01:21:03.589Z)

Verdict: **merge**. I verified all load-bearing claims locally against the real golden corpus.

Key verifications:

- **Scope**: diff is exactly 2 files (`backend/eval/ablation.py` +29, `tests/test\_ablation.py` +57), nothing unrelated touched.
- **Corpus partition**: `output/` has 11 golden files; `\_is\_fusion\_schema` splits them into exactly 6 fusion + 5 lean, and `load\_videos('output')` returns exactly the 6 fusion videos.
- **Bug is real**: pre-fix `load\_videos` (from `git show HEAD~1`) reads `c["shuttle"][fn]` unconditionally with no guard and `\_is\_fusion\_schema` didn't exist ? so the 5 lean files crash with `KeyError: 'shuttle'`.
- **Litmus preserved**: `test\_litmus\_reproduces\_gate\_a` RAN (not skipped) and PASSED, reproducing ?F1?0.074, 6/0 wins, p?0.031, CI [0.042, 0.104], seg 1.01/1.68.
- **Pre-LGTM gate**: full suite 848 passed / 5 skipped (the 5 are unrelated owner-box golden-trajectory tests), ruff clean, mypy clean.

No must-fix. The only substantive should-fix-adjacent point ? the `>=6` gate vs `==6` litmus assertion ? is a pre-existing latent fragility that does not fire today (corpus has exactly 6 fusion files, litmus intentionally pinned to n=6). Two should-fixes (litmus count-growth fragility, sibling `fusion\_compare`/`fusion\_audio` loaders with the same unguarded pattern) and four nits are all pre-existing or theoretical hardening, none blocking.

Relevant files (absolute):

- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\backend\eval\ablation.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test\_ablation.py`